

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No: 2

Roll no: 49 PRN No: 12414235 Name of Student: Saloni Tiwari

Subject Name : Deep Learning

Problem Statement:

Design and implement a Multilayer Perceptron (MLP) for classification of the Iris or Wine dataset, and evaluate its performance using accuracy and a confusion matrix.

Algorithm:

- Import the required libraries for data handling, visualization, model building, and evaluation.
- Load the Iris dataset and separate it into input features (X) and target labels (y).
- Display the dataset information, statistical summary, and class distribution.
- Check the dataset for any missing values.
- Split the dataset into training and testing sets using an 80:20 ratio.
- Create an MLP (Multilayer Perceptron) classifier with two hidden layers and ReLU activation.
- Train the MLP model using the training data.
- Predict the class labels for the test data.
- Calculate the model's accuracy.
- Generate and display the confusion matrix as a heatmap.
- Display the classification report showing precision, recall, F1-score, and support.
- Compare the actual and predicted class labels to evaluate the model's performance.

Code:

```
1] # import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
```

```

from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

2] # loading the dataset
iris = load_iris()

X = iris.data
y = iris.target

df = pd.DataFrame(X, columns=iris.feature_names)
df["Species"] = y

print(df.head())

3] # dataset info
print(df.info())

print(df.describe())

print(df["Species"].value_counts())

4] #checking missing values
print(df.isnull().sum())

5] #training and testing splitting
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

6] #create MLP model
mlp = MLPClassifier(
    hidden_layer_sizes=(10,10),
    activation='relu',
    solver='adam',
    max_iter=1000,
    random_state=42
)

7] #training the model
mlp.fit(X_train, y_train)

8] #predicitng
y_pred = mlp.predict(X_test)

9] #evaluating accuracy
accuracy = accuracy_score(y_test, y_pred)

```

```

print("Accuracy :", accuracy)

10] #making a confusion matrix
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,5))

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=iris.target_names,
    yticklabels=iris.target_names
)

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.title("Confusion Matrix")

plt.show()

11] #classification report
print(classification_report(
    y_test,
    y_pred,
    target_names=iris.target_names
))

12] #actual vs predicted values
results = pd.DataFrame({
    "Actual": y_test,
    "Predicted": y_pred
})

print(results.head(15))

```

Output:

1]

```
# import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

2]

```
# loading the dataset
iris = load_iris()

X = iris.data
y = iris.target

df = pd.DataFrame(X, columns=iris.feature_names)
df["Species"] = y

print(df.head())
```

...	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	\
0	5.1	3.5	1.4	0.2	
1	4.9	3.0	1.4	0.2	
2	4.7	3.2	1.3	0.2	
3	4.6	3.1	1.5	0.2	
4	5.0	3.6	1.4	0.2	
	Species				
0	0				
1	0				
2	0				
3	0				
4	0				

3]

```
#checking missing values
print(df.isnull().sum())

...  sepal length (cm)    0
      sepal width (cm)   0
      petal length (cm)  0
      petal width (cm)   0
      Species            0
      dtype: int64
```

4]

```
#training and testing splitting
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

5]

```
#training the model
mlp.fit(X_train, y_train)
```

MLPClassifier

MLPClassifier(hidden_layer_sizes=(10, 10), max_iter=1000, random_state=42)

6]

```
#evaluating accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy :", accuracy)

... Accuracy : 0.9333333333333333
```

7]

```
#making a confusion matrix
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,5))

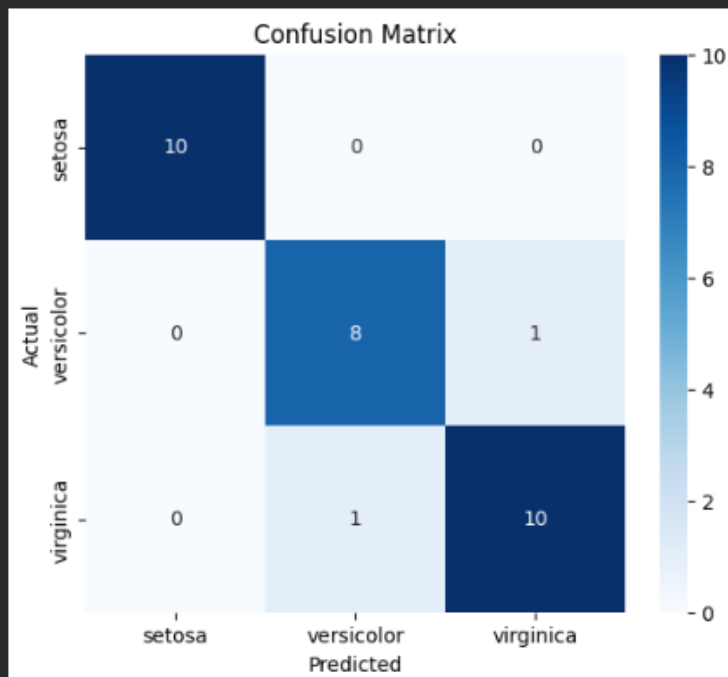
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=iris.target_names,
    yticklabels=iris.target_names
)

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.title("Confusion Matrix")

plt.show()
```



8]

```
#classification report
print(classification_report(
    y_test,
    y_pred,
    target_names=iris.target_names
))
```

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	0.89	0.89	0.89	9
virginica	0.91	0.91	0.91	11
accuracy			0.93	30
macro avg	0.93	0.93	0.93	30
weighted avg	0.93	0.93	0.93	30

9]

```
#actual vs predicted values
results = pd.DataFrame({
    "Actual": y_test,
    "Predicted": y_pred
})

print(results.head(15))
```

...	Actual	Predicted
0	1	1
1	0	0
2	2	2
3	1	1
4	1	1
5	0	0
6	1	1
7	2	2
8	1	2
9	1	1
10	2	2
11	0	0
12	0	0
13	0	0
14	0	0